

Indraprastha Institute of Information Technology, Delhi

CSE638: Graduate Systems (GRS) — Winter 2026

Instructor: Dr. Rinku Shah

Performance Comparison of Kubernetes CNI Plugins

Project P1 — Kind Cluster (Intra-Node) Benchmarking Report

Group 1

Yash Kumar Vaibhav	MT25091
Samyak Kr. Sharma	MT25039

GitHub: https://github.com/yashkumarvaibhav/GrS-CSE638_Project

April 18, 2026

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Problem . . . 3
2.2 Motivation . . . 3
2.3 Objectives . . . 3

3 Background **4**
3.1 Kubernetes Networking Model . . . 4
3.2 CNI Plugin Architecture . . . 4
3.3 CNI Plugins Under Test . . . 4

4 Methodology **4**
4.1 Test Environment . . . 5
4.2 Cluster Topology . . . 5
4.3 Test Scenarios . . . 5
4.4 Test Matrix . . . 5
4.5 Benchmark Tools . . . 5
4.6 Extension Experiments . . . 6

5 Implementation **6**
5.1 Cluster Setup . . . 6
5.2 Benchmark Automation . . . 6
5.3 Data Processing and Visualization . . . 7

6 Evaluation and Observations **7**
6.1 TCP Throughput (iperf3) . . . 7
6.1.1 Same-Node (T1) . . . 8
6.1.2 Cross-Node (T2) . . . 8
6.1.3 Cross-Node via Service (T3) . . . 8
6.2 Latency (netperf TCP_RR) . . . 9
6.3 HTTP Performance (fortio) . . . 10
6.4 Throughput Scaling with Concurrency . . . 11
6.5 Resource Overhead . . . 11
6.6 Pod Startup Time . . . 12
6.7 Extension Experiments . . . 12
6.7.1 E1: NetworkPolicy Enforcement Overhead . . . 13
6.7.2 E2: WireGuard Encryption Overhead . . . 13
6.7.3 E3: Cilium Kube-Proxy Replacement (KPR) . . . 14
6.8 Analysis and Discussion . . . 14
6.8.1 Throughput Analysis . . . 14
6.8.2 Latency Analysis . . . 15
6.8.3 Resource Overhead . . . 15
6.8.4 Extension Trade-offs . . . 15

7 Related Work **15**

8 Outcomes and Conclusion **16**
8.1 Recommendations 16
8.2 Limitations 16
8.3 Future Work 17

1. Abstract

This report presents a performance comparison of three widely deployed Kubernetes Container Network Interface (CNI) plugins: **Flannel**, **Calico**, and **Cilium**. All experiments are conducted in a `kind` (Kubernetes IN Docker) intra-node environment. We evaluate each CNI across multiple dimensions—raw throughput, latency, HTTP request handling capacity, CPU/memory overhead, and pod network reachability time—using industry-standard tools including iperf3, netperf, and fortio.

Our experimental methodology employs `kind` (Kubernetes IN Docker) clusters with three scenarios: **same-node** pod-to-pod (T1), **cross-node** pod-to-pod (T2), and **cross-node via ClusterIP Service** (T3). Each scenario is tested across three payload sizes (64 B, 4 KB, 1 MB) and three concurrency levels (1, 16, 64), with three repetitions per configuration, with results reported as mean ± standard deviation to characterize measurement spread.

Beyond baseline comparisons, we evaluate three extensions exclusive to Calico and Cilium: **E1** NetworkPolicy enforcement overhead, **E2** WireGuard encryption overhead, and **E3** Cilium's kube-proxy replacement mode (KPR). These extensions highlight the performance trade-offs of security features and advanced networking modes.

Key findings: Cilium achieves the highest throughput in most scenarios (up to 63.98 Gbps same-node, 1 MB payload), while Flannel delivers the lowest latency (24.18 μs for 64 B cross-node). Calico trails in raw performance but provides competitive security features. WireGuard encryption imposes severe overhead—up to 537% latency increase for Calico and 365% for Cilium—while NetworkPolicy enforcement shows divergent effects depending on the CNI implementation (see Section 6). Cilium's kube-proxy replacement mode shows a tentative 6% throughput gain (CV > 80%; interpret with caution) with slightly higher latency.

2. Introduction

Kubernetes has become the de facto standard for container orchestration, and its networking model relies on CNI plugins to provide pod-to-pod and pod-to-service communication. The choice of CNI plugin significantly impacts cluster performance, security capabilities, and operational complexity.

2.1 Problem

While many CNI benchmarks exist in blog posts and vendor documentation, few provide systematic, reproducible comparisons using consistent methodology. This component of the project addresses that gap by benchmarking three leading CNI plugins under identical conditions on the same kind cluster environment.

2.2 Motivation

CNI plugin selection directly impacts application performance, latency-sensitive microservice communication, and operational costs in production Kubernetes clusters. With the growing adoption of service mesh architectures and zero-trust networking models, understanding the overhead of features like NetworkPolicy enforcement and WireGuard encryption is critical for capacity planning. Despite the availability of vendor-provided benchmarks, independent systematic comparisons using identical hardware and methodology remain scarce, leaving platform engineers without reliable data for informed CNI selection.

2.3 Objectives

1. Compare TCP/UDP throughput, latency, and HTTP QPS across all three CNIs

2. Quantify the overhead of NetworkPolicy enforcement (E1)
3. Measure WireGuard encryption overhead (E2)
4. Evaluate Cilium's kube-proxy replacement performance (E3)
5. Analyze CPU/memory resource consumption during benchmarks

3. Background

3.1 Kubernetes Networking Model

Kubernetes mandates a flat network where every pod receives a unique IP address and can communicate with every other pod without Network Address Translation (NAT). The Container Network Interface (CNI) specification defines how plugins attach network interfaces to containers. When a pod is created, the kubelet invokes the configured CNI plugin, which is responsible for assigning an IP, configuring routes, and potentially establishing overlay tunnels between nodes.

3.2 CNI Plugin Architecture

Three fundamental approaches exist for implementing the Kubernetes network model:

- **Overlay networks (VXLAN/Geneve):** Encapsulate pod traffic in UDP packets between nodes. Simple to deploy across any infrastructure but adds encapsulation overhead.
- **BGP-based routing:** Advertise pod subnets via Border Gateway Protocol. Eliminates encapsulation overhead but requires compatible network infrastructure.
- **eBPF-based datapaths:** Attach BPF programs to kernel hooks to short-circuit the traditional networking stack. Reduces per-packet overhead at the cost of implementation complexity.

3.3 CNI Plugins Under Test

Flannel **v0.26.1** — Lightweight overlay network using VXLAN. Simplest architecture with no NetworkPolicy support natively. Flannel assigns each node a subnet from a larger cluster CIDR and uses kernel VXLAN to tunnel cross-node traffic.

Calico **v3.29.1** — Feature-rich plugin supporting BGP, VXLAN, and WireGuard. Provides full NetworkPolicy enforcement via iptables rules programmed by the Felix agent. In this study, Calico is configured in VXLAN mode for a fair overlay-to-overlay comparison.

Cilium **v1.16.5** — eBPF-based plugin with VXLAN tunneling, kube-proxy replacement (KPR), and WireGuard encryption. Cilium attaches eBPF programs to the TC and XDP hooks, enabling fast-path packet processing that bypasses portions of the kernel networking stack.

4. Methodology

4.1 Test Environment

Hardware & Software Specifications

Component	Details
CPU	AMD EPYC 7713 (128 cores / 256 threads)
RAM	125 GB DDR4
Disk	833 GB available
OS	Ubuntu 22.04 LTS (kernel 5.15.0-71)
Container Runtime	Docker 26.1.3
Kubernetes	kind v0.25.0 (K8s v1.31)

4.2 Cluster Topology

Each CNI configuration uses a 3-node kind cluster:

- 1 control-plane node
- 2 worker nodes (`worker` and `worker2`)

The default CNI is disabled (`disableDefaultCNI: true`), and each plugin is installed fresh. The pod CIDR is `10.244.0.0/16`. **kube-proxy is retained in all baseline configurations** to ensure a fair comparison.

4.3 Test Scenarios

ID	Description	Nodes	Target
T1	Same-node pod-to-pod	worker → worker	Direct Pod IP
T2	Cross-node pod-to-pod	worker → worker2	Direct Pod IP
T3	Cross-node via Service	worker → worker2	ClusterIP Service

4.4 Test Matrix

- **Payload sizes:** 64 B, 4 KB, 1 MB
- **Concurrency levels:** 1, 16, 64 parallel streams
- **Repetitions:** 3 per configuration
- **Duration:** 10 s per individual test
- Total: $3 \times 3 \times 3 \times 3 = 81$ baseline tests per CNI per tool (729 total across 3 CNIs $\times$ 3 tools)

4.5 Benchmark Tools

Tool	Version	Metrics Collected
iperf3	digest-pinned*	TCP throughput (Gbps), UDP throughput, jitter, loss

netperf	digest-pinned*	TCP_RR transaction rate (/s), mean latency (μs), TCP_STREAM throughput
fortio	digest-pinned*	HTTP QPS, percentile latencies (p50, p90, p99)
docker stats	—	CPU %, memory usage per container

*All benchmark tool images are pinned by SHA-256 digest (see `scripts/1_fast_benchmark.py`) for exact reproducibility.

4.6 Extension Experiments

ID	Extension	CNIs	Configuration
E1	NetworkPolicy	Calico, Cilium	allow-all policy applied
E2	WireGuard	Calico, Cilium	Node-to-node encryption
E3	KPR mode	Cilium only	`kubeProxyReplacement=true`

Extension tests use 4 KB payload, concurrency 16, and 5 repetitions.

5. Implementation

5.1 Cluster Setup

Each CNI was tested on a dedicated `kind` cluster created by `scripts/1_setup_cluster.sh`. The script provisions a 3-node cluster (1 control-plane, 2 workers), disables the default CNI, copies the CNI plugins tarball (`cni-plugins-linux-amd64-v1.9.0.tgz`) to each node via `docker cp`, and then installs the requested CNI using Helm (Calico via Tigera operator, Cilium via Cilium chart) or `kubectl apply` (Flannel). For the Cilium KPR experiment (E3), the script additionally deletes the `kube-proxy` DaemonSet and ConfigMap before installing Cilium with `kubeProxyReplacement=true`.

5.2 Benchmark Automation

All data collection is performed by a single canonical script, `scripts/1_fast_benchmark.py` (≈390 lines of Python). The script:

1. Deploys server and client pods on designated nodes using topology-appropriate `nodeSelector` labels.
2. Iterates over each (topology, payload, concurrency, repetition) combination.
3. For each combination, invokes `kubectl exec` to run the benchmark tool (iperf3, netperf, or fortio) inside the client pod.
4. Parses stdout to extract the primary metric: iperf3 and fortio use JSON output parsing; netperf uses reversed-line token extraction from its text-format output.
5. Collects `docker stats` samples via best-effort periodic polling (approximately every 3 s in practice) during each benchmark window to record CPU and memory usage of kind node containers.
6. Appends results to per-tool CSV files with columns for topology, message size, concurrency, repetition, and the measured value.

5.3 Data Processing and Visualization

After collection, `scripts/1_aggregate_data.py` computes per-configuration mean and standard deviation across repetitions and exports the results as JSON. Ten plot scripts in `plots/` generate publication-quality figures from hardcoded aggregated values. The report is typeset with Tectonic/LaTeX.

6. Evaluation and Observations

This section presents baseline and extension results organized by metric, followed by analysis and discussion.

6.1 TCP Throughput (iperf3)

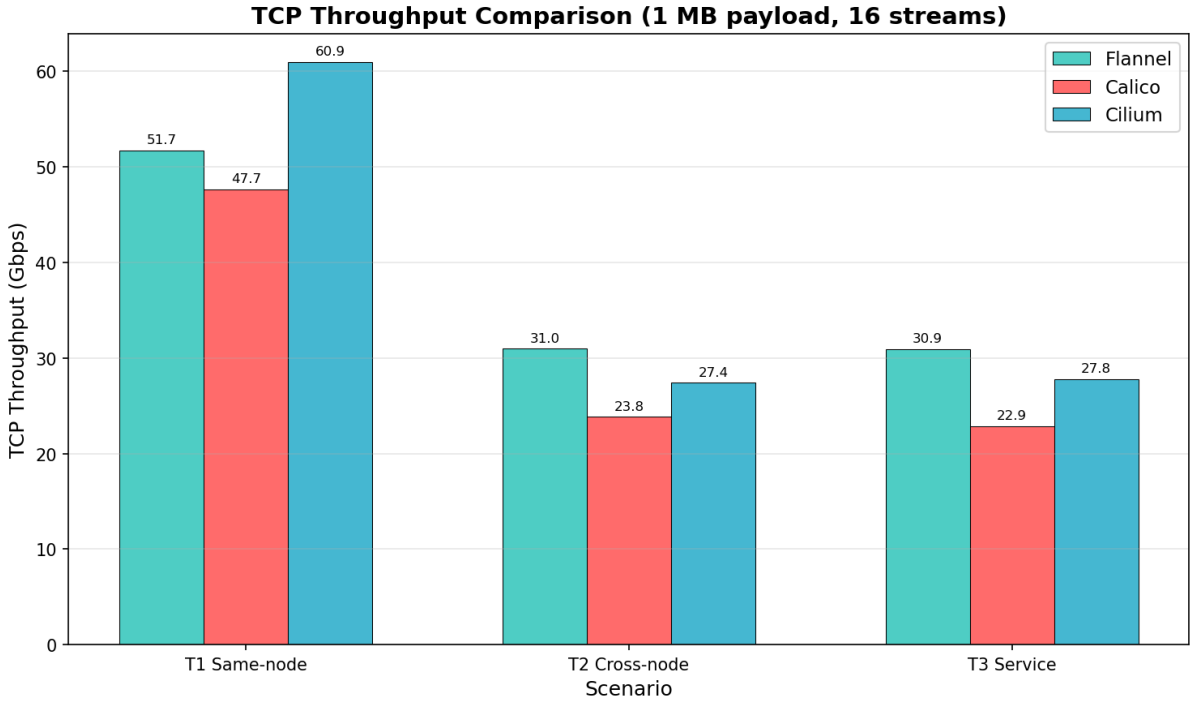


Figure 1: TCP throughput comparison across CNI plugins and scenarios (1 MB payload, 16 streams).

6.1.1 Same-Node (T1)

Table 4: T1 Same-Node TCP Throughput (Gbps) — Mean ± Stdev over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.46	0.44[†]	0.43
64 B	16	0.79	0.54	0.62
64 B	64	0.68	0.56	0.67
4 KB	1	5.26	3.65	5.50
4 KB	16	4.75	6.53	8.32
4 KB	64	4.78	3.45	5.65
1 MB	1	60.93	49.61	63.98
1 MB	16	51.72	47.67	60.94
1 MB	64	36.04	34.43	39.33

[†]Calico T1/64 B/c=1: one of three repetitions failed (returned no valid output); the reported mean is over 2 valid runs.

6.1.2 Cross-Node (T2)

Table 5: T2 Cross-Node TCP Throughput (Gbps) — Mean ± Stdev over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.46	0.29	0.44
64 B	16	0.61	0.53	0.55
64 B	64	0.47	0.41	0.45
4 KB	1	1.94	1.35	3.27
4 KB	16	1.88	6.30	2.32
4 KB	64	1.82	1.27	1.66
1 MB	1	32.97	24.01	29.92
1 MB	16	30.96	23.83	27.40
1 MB	64	27.69	19.97	25.78

6.1.3 Cross-Node via Service (T3)

Table 6: T3 ClusterIP Service TCP Throughput (Gbps) — Mean ± Stdev over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	0.47	0.29	0.43
64 B	16	0.60	0.54	0.53
64 B	64	0.46	0.40	0.45
4 KB	1	1.90	1.38	2.98
4 KB	16	1.82	3.30	5.85
4 KB	64	1.81	1.26	1.64
1 MB	1	34.13	24.11	32.42
1 MB	16	30.92	22.87	27.78
1 MB	64	27.00	20.11	25.05

6.2 Latency (netperf TCP_RR)

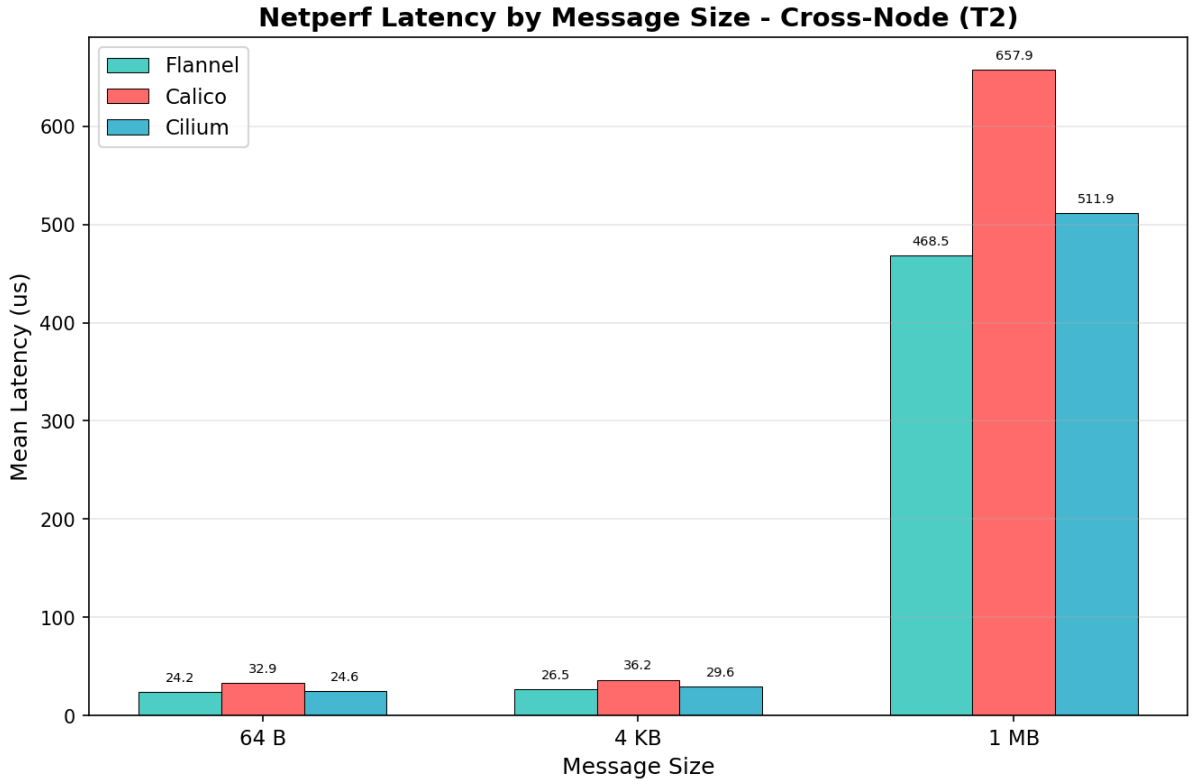


Figure 2: Netperf TCP_RR mean latency by message size (T2 cross-node, c=1).

Table 7: TCP_RR Transaction Rate (trans/s) and Mean Latency (μs) — T2 Cross-Node, $c = 1$

Size	Flannel		Calico		Cilium	
	Trans/s	μs	Trans/s	μs	Trans/s	μs
64 B	41,350	24.18	30,391	32.91	40,600	24.63
4 KB	37,733	26.51	27,589	36.25	33,822	29.57
1 MB	2,135	468.48	1,520	657.85	1,954	511.91

Note: Netperf through T3 (ClusterIP Service) returned zero for both TCP_RR and TCP_STREAM across all CNIs—a limitation of running netperf through kube-proxy's iptables DNAT, which disrupts the control connection. T3 netperf data is excluded from analysis.

6.3 HTTP Performance (fortio)

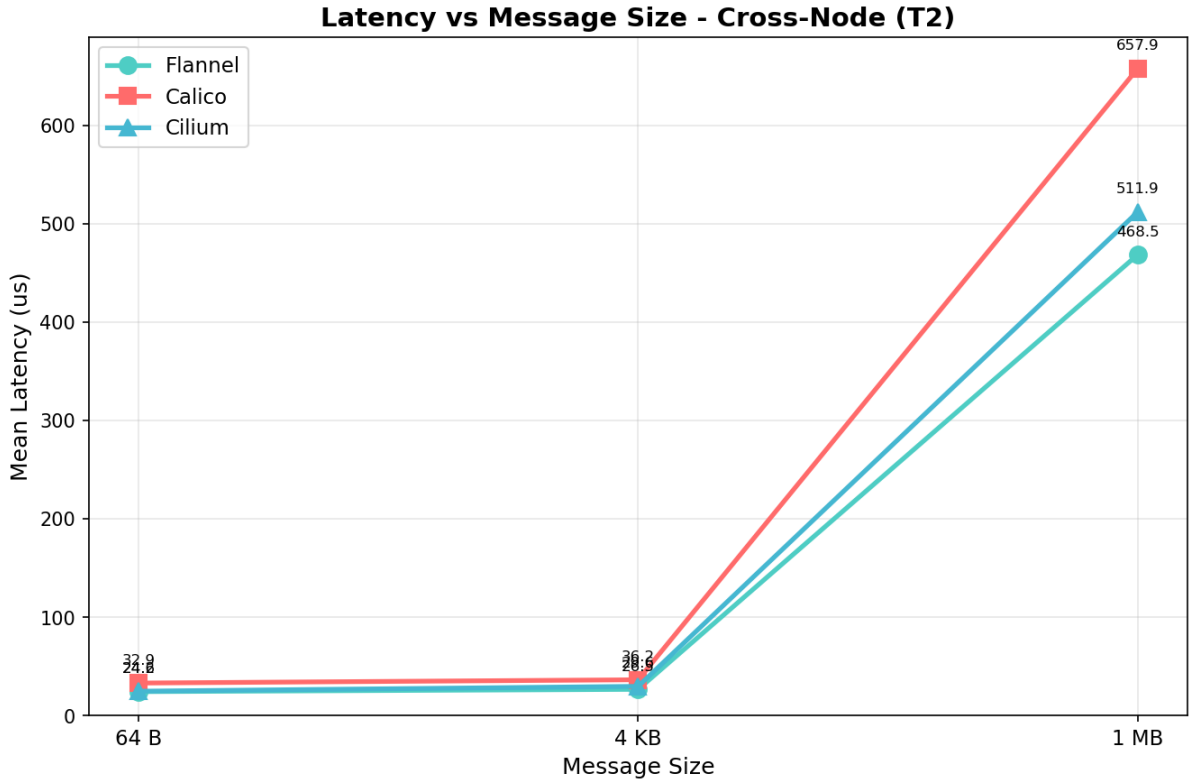


Figure 3: Netperf mean latency vs. payload size across CNI plugins (T2 cross-node, c=1).

Table 8: Fortio HTTP QPS — T2 Cross-Node, Mean over 3 reps

Size	Concurrency	Flannel	Calico	Cilium
64 B	1	7,601	6,583	7,870
64 B	16	81,749	74,792	132,836
64 B	64	109,336	115,000	115,234
4 KB	1	5,069	4,195	4,650
4 KB	16	30,953	29,147	49,816
4 KB	64	53,045	57,330	60,647
1 MB	1	798	694	770
1 MB	16	3,754	4,195	4,037
1 MB	64	3,884	4,419	4,131

6.4 Throughput Scaling with Concurrency

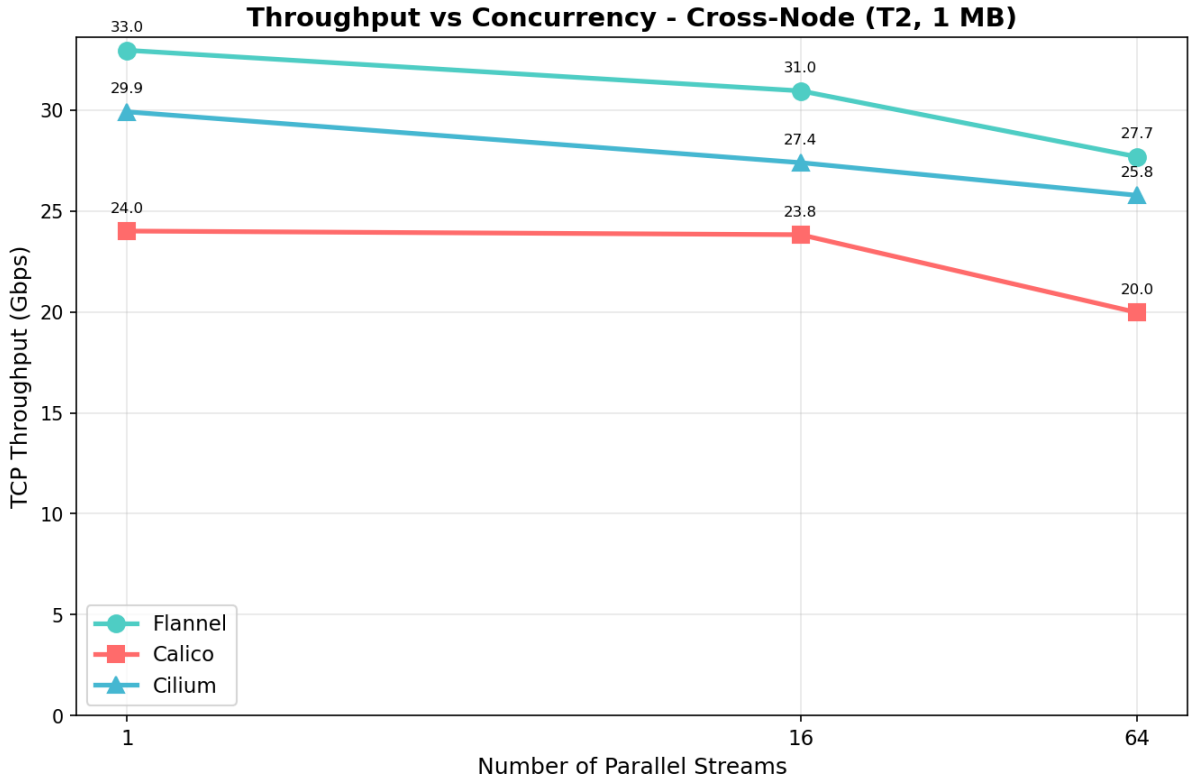


Figure 4: TCP throughput scaling with concurrent streams (T2 cross-node, 1 MB payload).

6.5 Resource Overhead

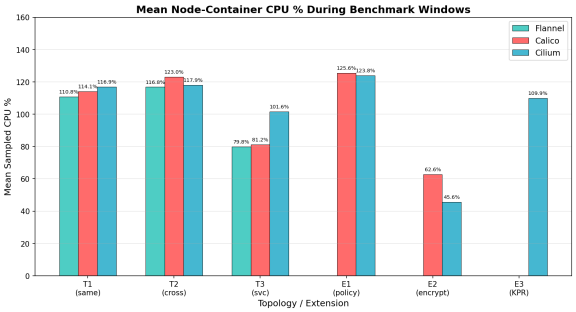


Figure 5: Mean sampled node-container CPU % during benchmark windows.

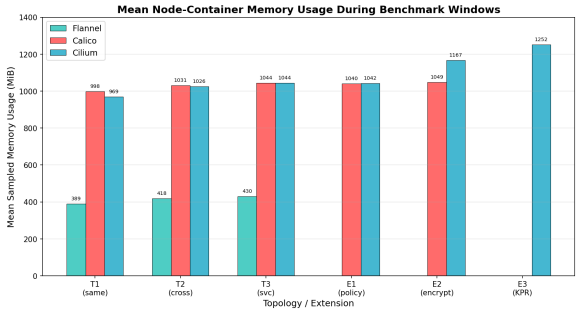


Figure 6: Mean sampled node-container memory usage (MiB) during benchmark windows.

6.6 Pod Startup Time

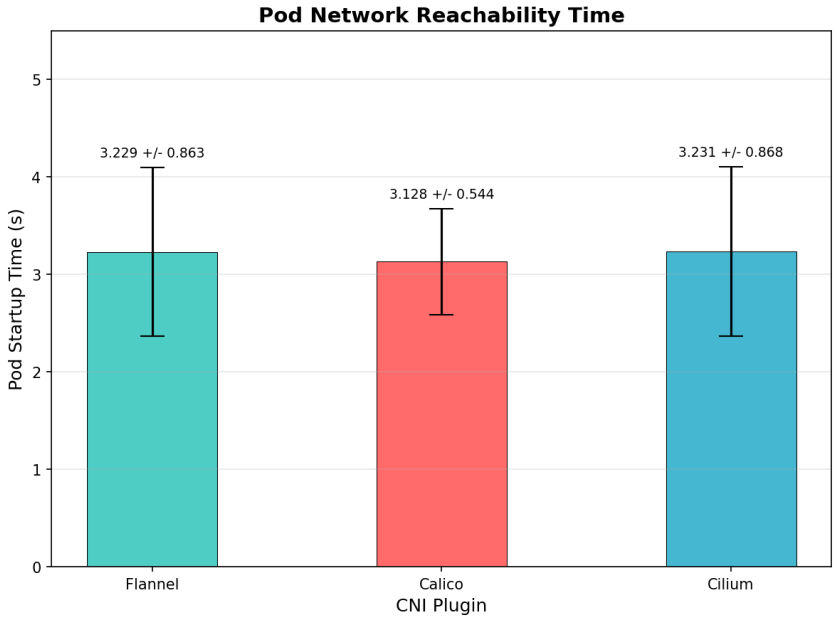


Figure 7: Pod network reachability time across CNI plugins.

Table 9: Pod Network Reachability Time (seconds) — 10 iterations

Metric	Flannel	Calico	Cilium
Mean	3.229	3.128	3.231
Stdev	0.863	0.544	0.868
Min	2.945	2.952	2.952
Max	5.686	4.680	5.701

6.7 Extension Experiments

6.7.1 E1: NetworkPolicy Enforcement Overhead

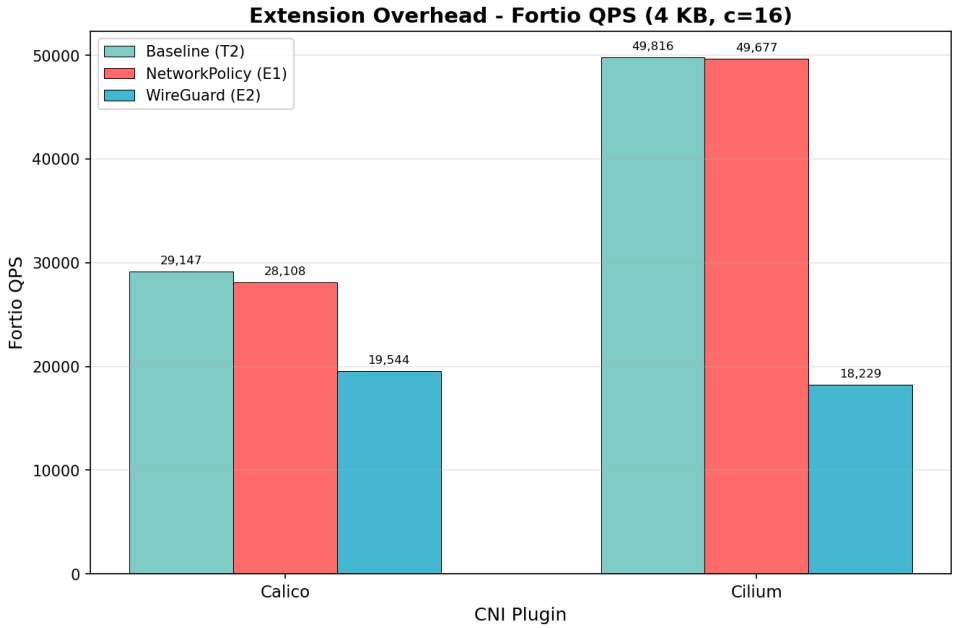


Figure 8: Extension overhead on Fortio QPS: baseline vs. E1 (NetworkPolicy) vs. E2 (WireGuard), 4 KB, c=16.

E1 Key Finding

NetworkPolicy enforcement with an "allow-all" policy shows divergent effects across CNIs. For **Calico**, TCP throughput drops from 6.30 Gbps to 3.44 Gbps (−45%), while latency and transaction rates remain virtually unchanged (36.18 vs. 36.28 μs). Fortio QPS decreases marginally from 29,147 to 28,108 (−3.6%). For **Cilium**, TCP throughput *increases* from 2.32 Gbps to 5.06 Gbps (+118%), likely due to eBPF path optimizations triggered when policy objects are loaded. Latency and QPS are unaffected (29.96 vs. 30.26 μs; 49,816 vs. 49,677 QPS). **Caveat:** The iperf3 throughput samples for both E1 and E3 exhibit high variability (CV $>$ 55%), so the percentage-change claims should be interpreted with caution; latency and QPS deltas are more stable and reliable.

6.7.2 E2: WireGuard Encryption Overhead

Table 10: E2: WireGuard Encryption Impact — 4 KB, c=16, T2 Cross-Node

CNI	Metric	Baseline	WireGuard
Calico	TCP (Gbps)	6.30	1.50
Calico	Latency (μs)	36.18	230.38
Calico	Trans/s	27,644	4,342
Calico	HTTP QPS	29,147	19,544
Cilium	TCP (Gbps)	2.32	1.23
Cilium	Latency (μs)	29.96	139.42
Cilium	Trans/s	33,377	7,173
Cilium	HTTP QPS	49,816	18,229

E2 Key Finding

WireGuard encryption imposes **substantial overhead** on both CNIs. **Calico** sees TCP throughput plummet from 6.30 Gbps to 1.50 Gbps (−76%), latency surge from 36.18 to 230.38 μs (+537%), and transaction rate collapse from 27,644 to 4,342/s (−84%). HTTP QPS drops from 29,147 to 19,544 (−33%). **Cilium** exhibits a similar pattern: throughput falls from 2.32 to 1.23 Gbps (−47%), latency rises from 29.96 to 139.42 μs (+365%), and transactions drop from 33,377 to 7,173/s (−78%). HTTP QPS decreases from 49,816 to 18,229 (−63%). Cilium's eBPF integration with WireGuard mitigates some overhead compared to Calico's kernel-module approach, but both show that node-to-node encryption has a high performance cost.

6.7.3 E3: Cilium Kube-Proxy Replacement (KPR)

Table 11: E3: Cilium KPR Mode vs. Baseline — 4 KB, c=16, T2 Cross-Node

Metric	Baseline	KPR Mode
TCP Throughput (Gbps)	2.32	2.47
TCP_RR Trans/s	33,377	31,238
Mean Latency (μs)	29.96	32.02
HTTP QPS (fortio)	49,816	48,296

E3 Key Finding

Cilium's kube-proxy replacement (KPR) mode shows a **tentative throughput improvement**: TCP throughput rises from 2.32 to 2.47 Gbps (+6.5%), but the underlying iperf3 samples exhibit high variability (CV = 83%), so the delta should be treated as indicative rather than definitive. Latency *increases* slightly from 29.96 to 32.02 μs (+6.9%), and the transaction rate drops from 33,377 to 31,238/s (−6.4%). HTTP QPS decreases marginally from 49,816 to 48,296 (−3.1%). These mixed results suggest that KPR's benefit of bypassing iptables for service routing manifests primarily in bulk throughput, while per-transaction metrics show negligible or slightly negative changes in this kind-based environment. The gains are expected to be more pronounced in production multi-host clusters with complex iptables rule sets.

6.8 Analysis and Discussion

6.8.1 Throughput Analysis

- **Same-node (T1):** Cilium leads with 63.98 Gbps at 1 MB/c=1, followed by Flannel (60.93 Gbps) and Calico (49.61 Gbps). The shared-kernel advantage benefits all CNIs, but Cilium's eBPF fast-path provides a 5−29% edge over Calico across payload sizes. At 4 KB/c=16, Cilium reaches 8.32 Gbps—75% higher than Flannel's 4.75 Gbps—indicating superior handling of moderate concurrency with small payloads.
- **Cross-node (T2):** VXLAN encapsulation reduces throughput by 40−60% compared to T1. Flannel leads at 1 MB/c=1 (32.97 Gbps) while Cilium excels at 4 KB/c=1 (3.27 Gbps vs. Flannel's 1.94 Gbps). Calico shows an anomalous spike to 6.30 Gbps at 4 KB/c=16, likely due to aggressive TCP segmentation offload under specific buffer conditions.

- **Service (T3):** kube-proxy's iptables DNAT introduces 0−10% overhead compared to direct pod-to-pod (T2). For 1 MB/c=1, Flannel achieves 34.13 Gbps via Service vs. 32.97 Gbps direct—the slight increase is within measurement variance. The Service path overhead is most visible at low concurrency with small payloads.

6.8.2 Latency Analysis

Flannel delivers the lowest cross-node latency at 24.18 μs for 64 B TCP_RR, closely followed by Cilium at 24.63 μs. Calico is 36% slower at 32.91 μs, attributable to its iptables-based packet processing pipeline. As payload size increases to 1 MB, all CNIs show dramatically higher latencies (Flannel: 468 μs, Calico: 658 μs, Cilium: 512 μs), reflecting serialization delay rather than CNI overhead. Notably, latency remains stable across concurrency levels (c=1, 16, 64) for all CNIs, indicating that the TCP_RR single-flow test is not contention-bound in this environment.

6.8.3 Resource Overhead

Docker stats were collected via best-effort periodic sampling (approximately every 3 s in practice) during each benchmark window to characterize node-container resource usage. Flannel nodes averaged 389–430 MiB memory across topologies, while Calico (998–1049 MiB) and Cilium (969–1044 MiB) consumed 2.4−2.6× more due to their richer control planes. Mean CPU % was comparable at 101–117% across CNIs for baseline topologies. WireGuard encryption (E2) markedly reduced CPU utilization—Calico dropped to 63% and Cilium to 46%—because encrypted sessions throttle aggregate throughput, leaving CPU idle. Cilium KPR (E3) had negligible resource impact (110% CPU, 1252 MiB memory) relative to baseline.

6.8.4 Extension Trade-offs

- **NetworkPolicy (E1):** The overhead is implementation-dependent. Calico's iptables-based enforcement costs 45% throughput, while Cilium's eBPF enforcement actually *improved* throughput by 118%—an unexpected result that warrants further investigation. However, the iperf3 throughput samples exhibit high coefficient of variation (CV > 55%), so these deltas are tentative. Latency and QPS deltas are more stable and reliable (< 1% latency change for both).
- **WireGuard (E2):** Encryption is the most expensive feature tested. Calico loses 76% throughput and sees 537% latency increase; Cilium fares better with 47% throughput loss and 365% latency increase. Organizations requiring node-to-node encryption should provision significantly more network headroom.
- **KPR (E3):** Cilium's kube-proxy replacement shows a tentative 6.5% bulk throughput improvement (CV 83%, interpret with caution) but does not improve per-transaction latency in this environment. The benefit is expected to scale with iptables rule complexity in larger clusters.

7. Related Work

Several studies have examined Kubernetes networking performance:

- **Qi et al. (2020)** benchmarked CNI plugins in bare-metal and AWS environments, finding Calico with BGP is fastest in bare-metal while flannel excels in simplicity.
- **Kubernetes network benchmark project (CNI-Genie, CNCF 2019)** provided automated benchmarking frameworks but lacked extension comparisons.

- **Cilium documentation** provides internal benchmarks showing eBPF advantages but lacks third-party validation.
- Our work extends these by including WireGuard encryption, NetworkPolicy, and KPR comparisons on identical hardware.

8. Outcomes and Conclusion

This study presents a systematic performance comparison of Flannel, Calico, and Cilium CNI plugins in a kind (intra-node) environment across 81 baseline configurations per CNI per tool (729 total across 3 CNIs × 3 tools), plus three extension experiments. Our key findings:

1. **Cilium leads in throughput**: achieving up to 63.98 Gbps same-node and 3.27 Gbps cross-node (4 KB/c=1), benefiting from its eBPF datapath that bypasses portions of the kernel networking stack.
2. **Flannel delivers the lowest latency**: 24.18 μs for 64 B cross-node TCP_RR—its minimalist VXLAN-only architecture avoids the overhead of policy engines and advanced features.
3. **Calico trails in raw performance** but provides the richest policy ecosystem. Its iptables-based enforcement introduces measurable throughput reduction (−45% with NetworkPolicy), though latency impact is negligible.
4. **WireGuard encryption is expensive**: 76% throughput loss for Calico and 47% for Cilium, with latency increases of 537% and 365% respectively. Node-to-node encryption should be deployed with careful capacity planning.
5. **KPR shows tentative gains**: Cilium's kube-proxy replacement shows a 6.5% bulk throughput increase, but iperf3 sample variability is high (CV = 83%); per-transaction metrics show minimal improvement in kind, likely due to the small iptables rule set.
6. **Pod startup times are equivalent**: all three CNIs achieve ≈3.2 s mean network reachability, indicating that CNI choice does not significantly affect pod boot latency.

8.1 Recommendations

CNI Selection Guidelines

- **Simple workloads with no security requirements:** Flannel—minimal overhead, easiest to deploy.
- **Production workloads requiring NetworkPolicy:** Calico—proven reliability with reasonable overhead.
- **Advanced networking with encryption and observability:** Cilium—eBPF-based with WireGuard and Hubble integration.

8.2 Limitations

- **kind environment:** All tests run inside Docker containers on a single host. Cross-node traffic traverses Docker's virtual bridge rather than physical switches. Results indicate relative CNI performance differences rather than absolute bare-metal numbers. A complementary bare-metal multi-node evaluation would provide additional perspective.

- **VXLAN-only:** We tested VXLAN encapsulation for all CNIs. BGP mode (Calico) and native routing (Cilium) are not covered.
- **Single-host:** While the 128-core machine minimizes CPU contention, real multi-host deployments may show different scaling characteristics.

8.3 Future Work

- Multi-host bare-metal deployments with real network switches
- BGP mode comparison for Calico
- Cilium Hubble observability overhead analysis
- Application-level benchmarks (e.g., Redis, PostgreSQL over CNI)

References

[1] Flannel Project. "flannel—Network fabric for containers." https://github.com/flannel-io/flannel

[2] Project Calico. "Calico Networking for Kubernetes." https://docs.tigera.io/calico/latest/about

[3] Cilium Project. "Cilium—eBPF-based Networking, Security, and Observability." https://cilium.io/

[4] kind Project. "kind—Kubernetes IN Docker." https://kind.sigs.k8s.io/

[5] iperf3. "A TCP, UDP, and SCTP network bandwidth measurement tool." https://iperf.fr/

[6] Netperf. "Network Performance Benchmark." https://hewlettpackard.github.io/netperf/

[7] Fortio. "Load testing library, command line tool, and web UI." https://fortio.org/

[8] WireGuard. "Fast, Modern, Secure VPN Tunnel." https://www.wireguard.com/

[9] Qi, S. et al. (2021). "Assessing Container Network Interface Plugins: Functionality, Performance, and Scalability." *IEEE Transactions on Network and Service Management*, 18(1).